# JC2001: Introduction to Software Engineering

## Lecture 1: Introduction & Course Overview

**Lecture Schedule:** Week 1, Day 1


Dr Zhenhua Yu zhenhua.yu@abdn.ac.uk

Dr Riazul Islam riazul.islam@abdn.ac.uk

Dr Shahzad Mumtaz shahzad.mumtaz@abdn.ac.uk

# Course Coordinator

**Dr Shahzad Mumtaz**

shahzad.mumtaz@abdn.ac.uk



# Course Delivery Team


**Dr Zhenhua Yu**

zhenhua.yu@abdn.ac.uk





**Dr Riazul Islam**

riazul.islam@abdn.ac.uk





**Dr Shahzad Mumtaz**

shahzad.mumtaz@abdn.ac.uk




# Outline

- What is a Software Product?
- What is Software Engineering?
- Challenges of software product development

} Why do we need to study software engineering?

- Course overview, learning objectives, and assessment

# What is a Software Product?







UNIVERSITY OF ABERDEEN

# How many lines of code are there…

| | |
|---|---|
| in Google Chrome? | 6 million |
| in Windows 10? | 50 million |
| in Microsoft Office? | 4000 million |
| in Boeing 787? | 7 million |
| in Facebook? | 60 million |



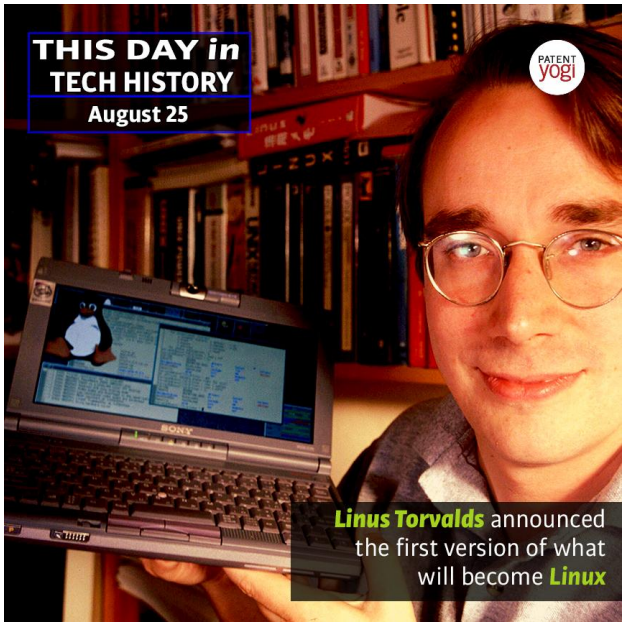
Software products

# What is Software as a Product?

**Software Product** – **Software Product** or **Software as a Product (SaaP)** is made to be sold to users.

**Prototype software** – Independent ‘in-house programs’ developed by few (one or two) people may implement some exciting features, but these are usually not sold to users. These kinds of software are known as **prototype software** (or sometimes also referred to as 'homemade software'/ 'amateur software'), but not products.

# Famous examples of prototype software




The 1st edition of Linux was developed by Linus Torvalds in 1991. It was small and simple, developed by only one person, so it is considered prototype software.

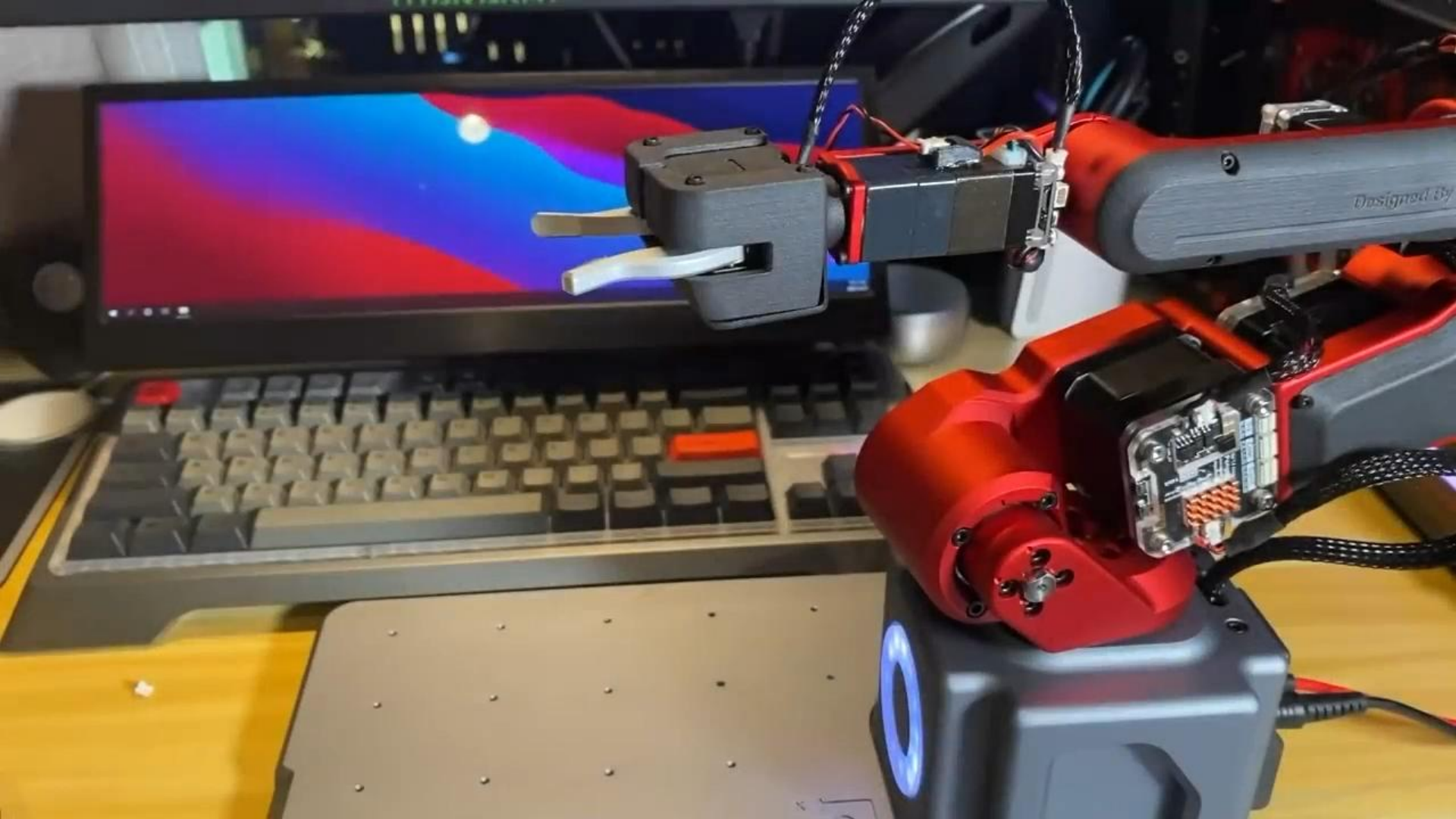
Today, however, Linux is continuously being developed and maintained by the *Free Software Community* and *Linux User Group*. The size of Linux has grown significantly, and it is now considered a software product.

# Famous examples of prototype software




A robotic arm

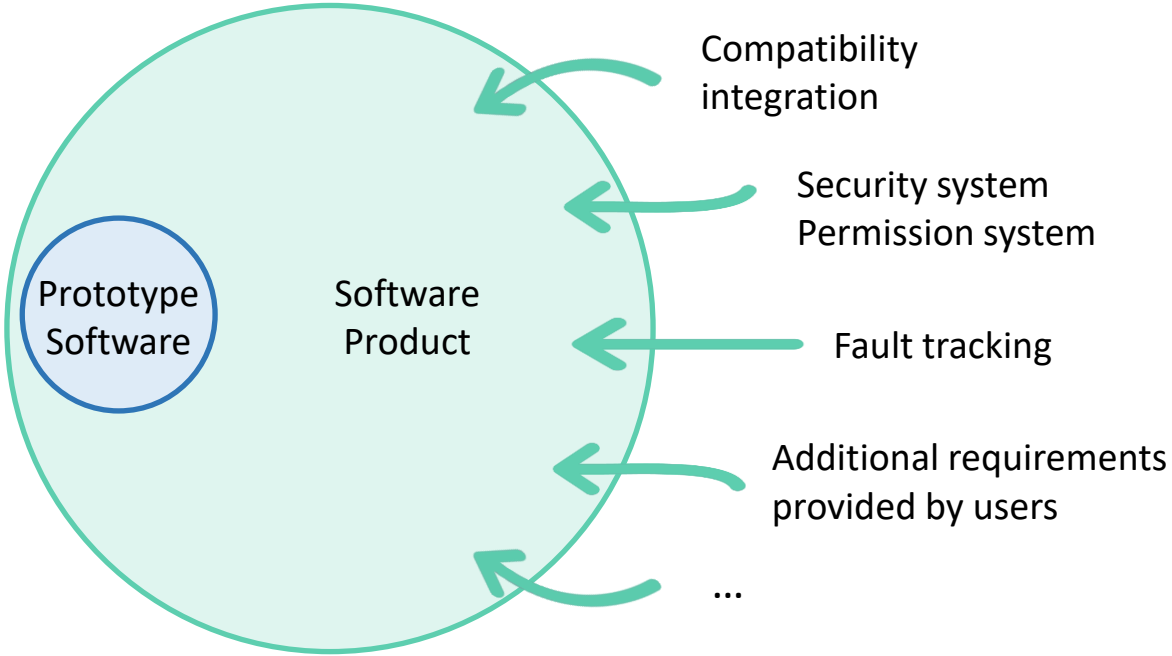
Huijun Zhi (2021)



# Prototype software vs. Software Product

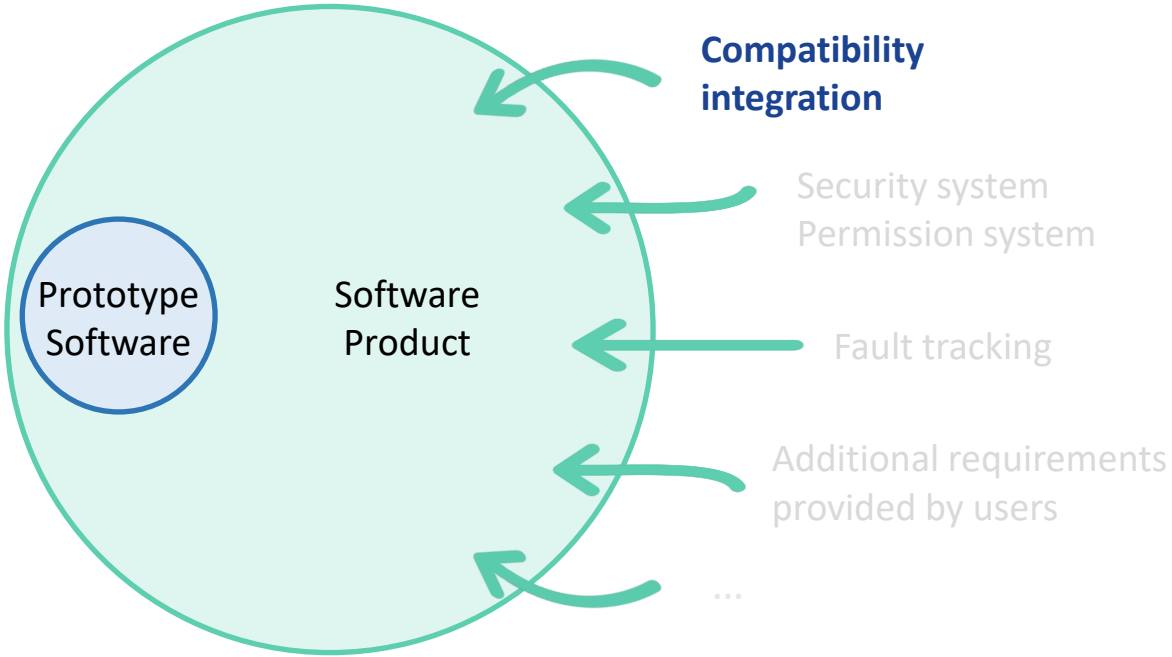
| Prototype Software | Software Products |
| --- | --- |
| • Developed for yourself (you know what you want) | • Developed for users (you need to find out what the users think) |
| • Implements core/exciting features only | • Implements core features, as well as additional accessibility features |
| • No worry for software security | • Software security is very important! |
| • No need to write technical documentation and instructions | • A lot of documentation is required: for inter-team communication, communication with users, and software instructions |

# What is Software as a Product?




Amount of code of a product software vs. its prototype

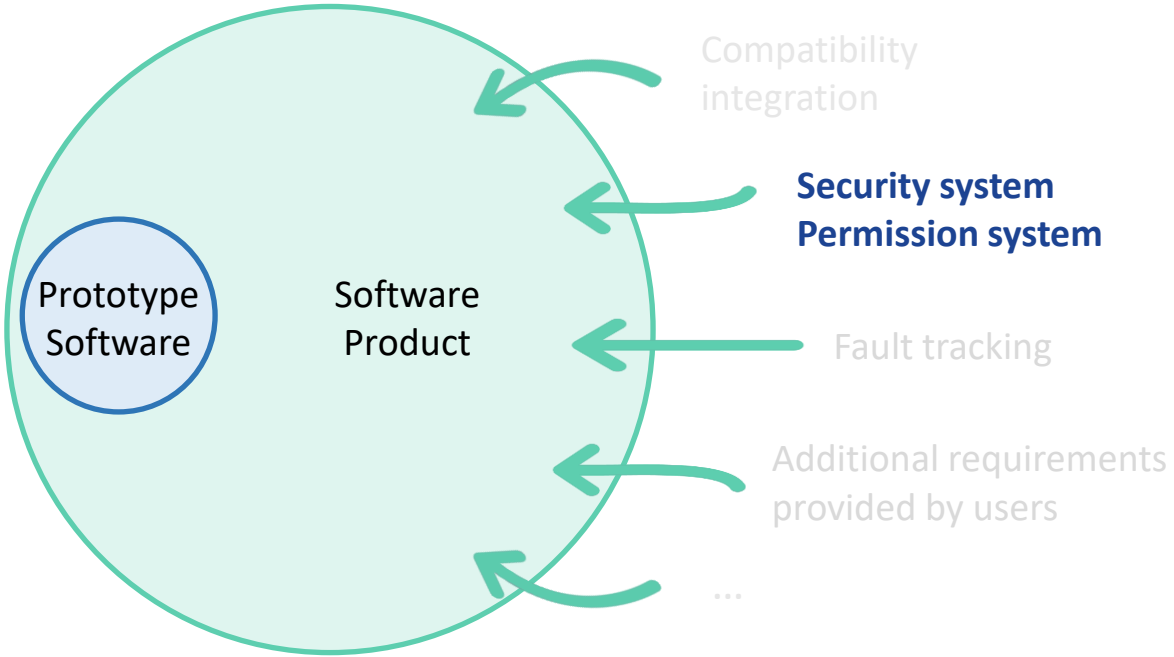
# What is Software as a Product?




Amount of code of a product software vs. its prototype

Software products should work with multiple operation systems.

# What is Software as a Product?



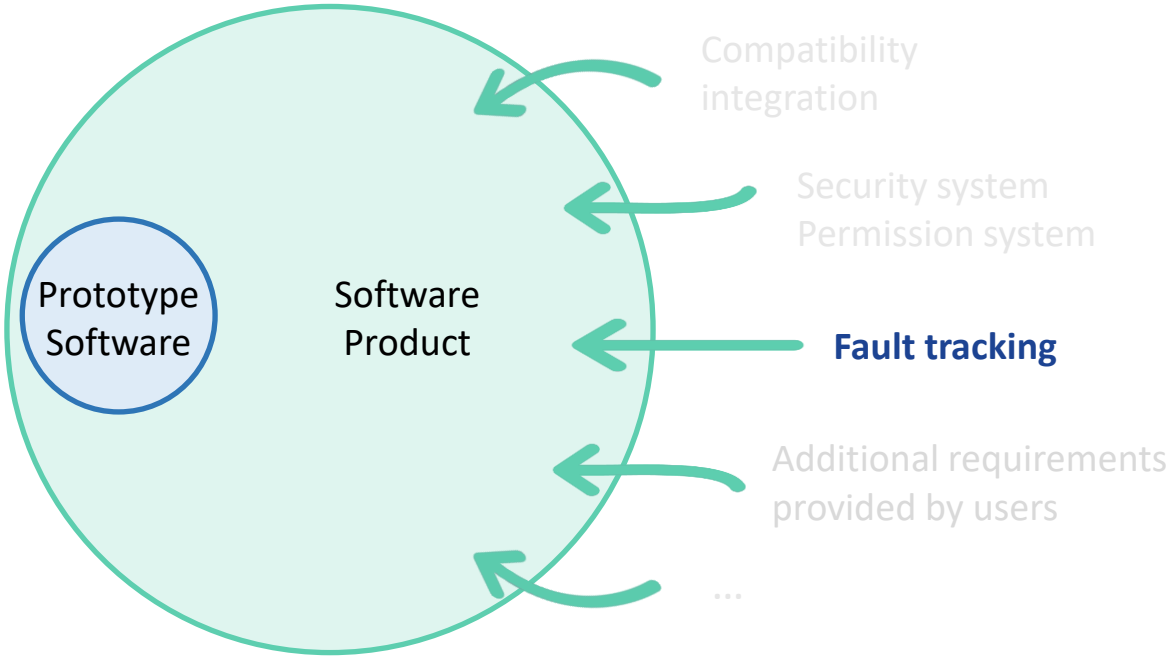

Amount of code of a product software vs. its prototype

Users and other people may attack your software for many reasons.

For example:

- To steal user data
- To access your background API
- Fraudulent use
- Pirating

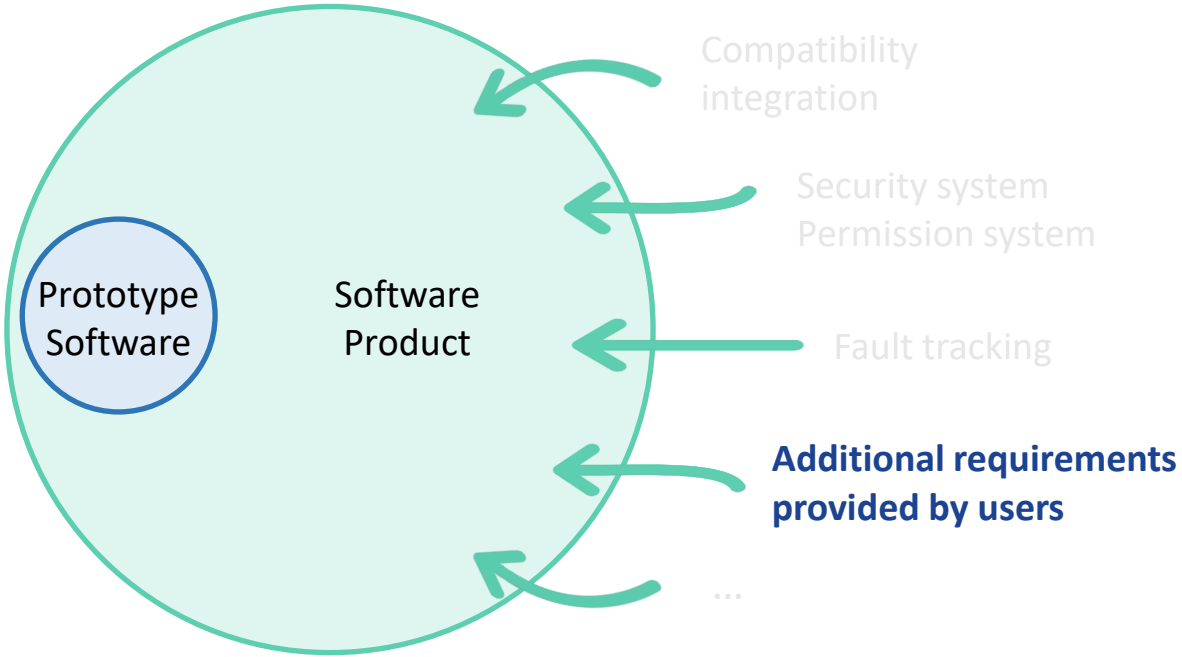
# What is Software as a Product?




Amount of code of a product software vs. its prototype

Track and collect bugs from user side.

# What is Software as a Product?

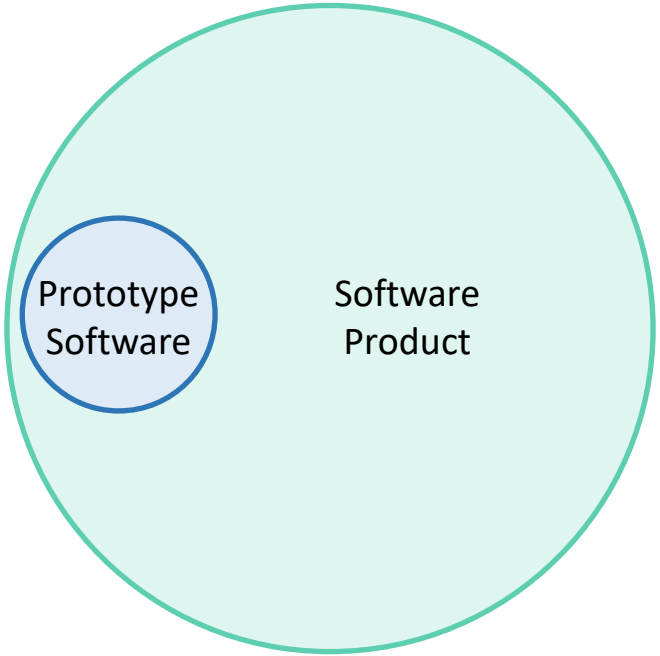



Amount of code of a product software vs. its prototype

Users may require customised features.

(If you are in a programming team, you may regard the project leader as your user)

# What is Software as a Product?

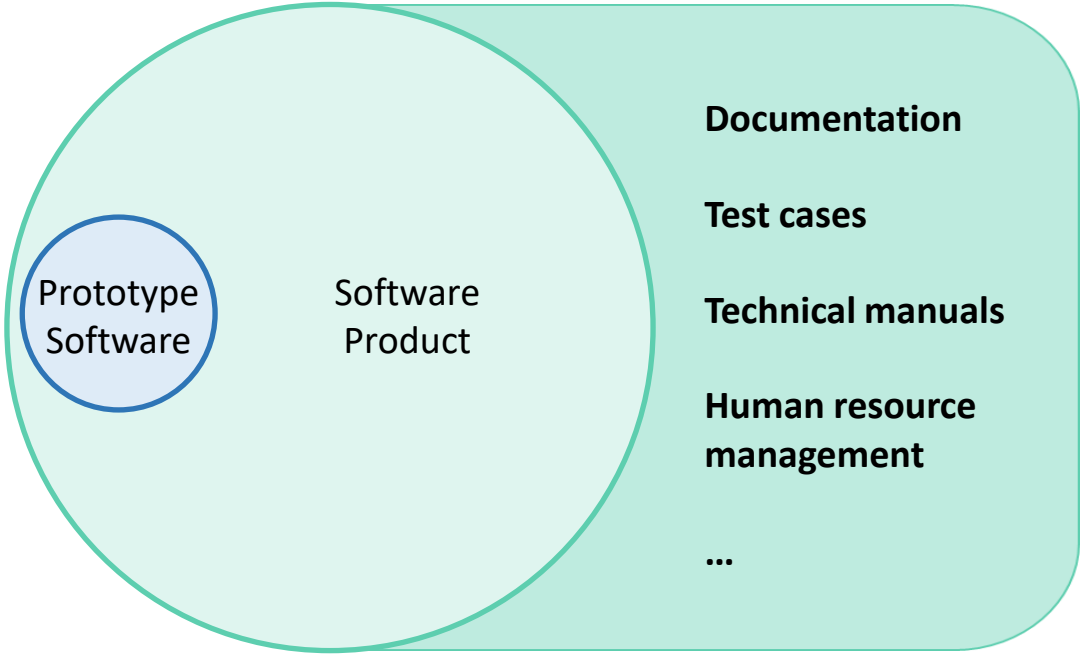



Amount of code of a product software vs. its prototype

When a prototype software evolves to a software product, many efforts should be made.

A software product has **a lot more code** than its prototype.

# What is Software as a Product?




Amount of code of a product software vs. its prototype

Beyond the programming, software product development includes many other human labour costs.

Software product development is far more difficult than prototyping.

# Most applications are products!

- We tend to develop software as an ongoing application
- Few people now build software as a one-off project.
- Instead, it is an ongoing application that never finishes being developed.

# What is Software Engineering?






1495 UNIVERSITY OF ABERDEEN

# What is Software Engineering?

**Software Engineering** is a collection of concepts, methods, and tools which focuses on the techniques to build **large** and **complex** software.

- **Large** means that it requires many people working together. We need to learn how to organise people to work together.
- **Complex** means that the software has multiple components which affect each other. We need to learn how to architect software and reduce its complexity.

# What is Software Engineering?

**Software Engineering** helps with the production of:

- a **high quality** software system
- with a given **budget**
- before a given **deadline**
- whilst **requirement changes** occur

# What is Software Engineering?

**Software Engineering** helps with the production of:

- a **high quality** software system

**Case: Therac-25**

- Therac-25 was a computer-controlled radiation therapy machine produced by Atomic Energy of Canada Limited (AECL) in 1982.
- Because of concurrent programming errors (also known as race conditions), it sometimes gave its patients radiation doses that were hundreds of times greater than normal.
- Patients were therefore death or serious injured.



# What is Software Engineering?

**Software Engineering** helps with the production of a high quality system:

- with a given **budget**
- before a given **deadline**

In 1995, the Standish Group research institute took 8,000 software projects in the United States as a survey sample. The survey results showed that 84% of software plans could not be completed within the set time and budget, and more than 30% of the projects were cancelled during implementation.

The average project budget over 189%.

# What is Software Engineering?

**Software Engineering** helps with the production of a high quality system, with a given budget, before a given deadline:

- whilst **requirement changes** occur

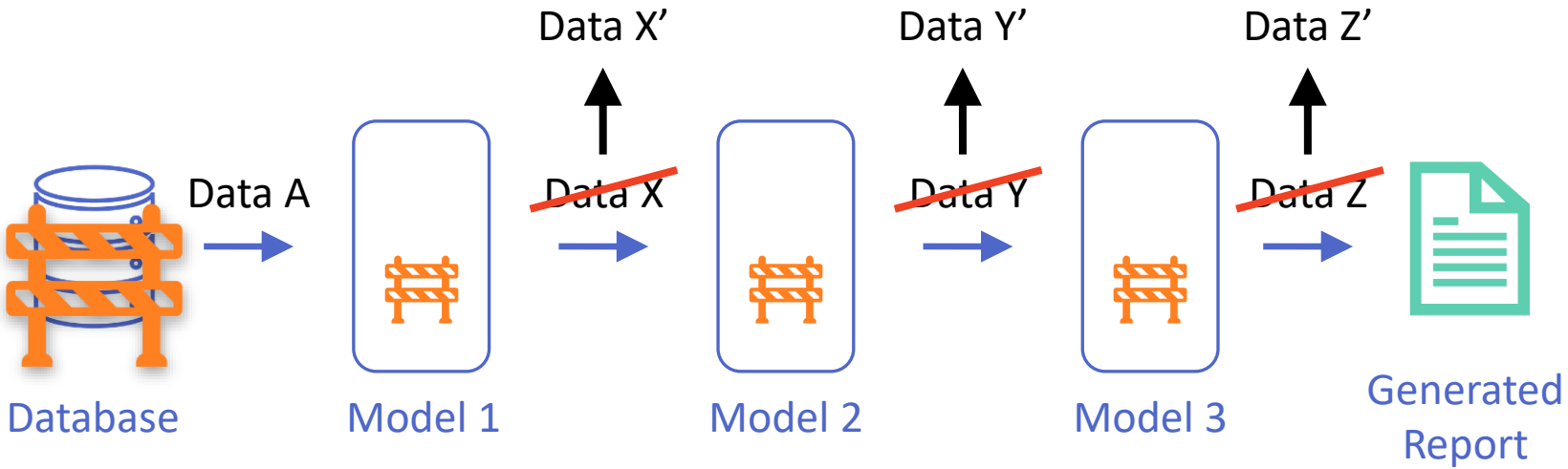
When developing software, requirements change frequently because you can always get feedback from various sources and need to modify the software based on that feedback.

However, many minor requirement changes to functionality may lead to substantial code rewrites and likely delays in software delivery.

# What is Software Engineering?

**Software Engineering** helps with the production of a high quality system, with a given budget, before a given deadline:

- whilst **requirement changes** occur



# What is Software Engineering?

## Typically, software products have 6 development processes:

**Planning –** decide what software and features to develop

**Requirements Capture –** collect requirements from experts & users.

**Design & Analysis –** design the software architecture and the overall structure of the program.

**Implementation –** coding

**Testing –** test the modules of the software and test the software as a whole to find bugs.

**Maintenance –** fix bugs reported by users



# Challenges of software product development



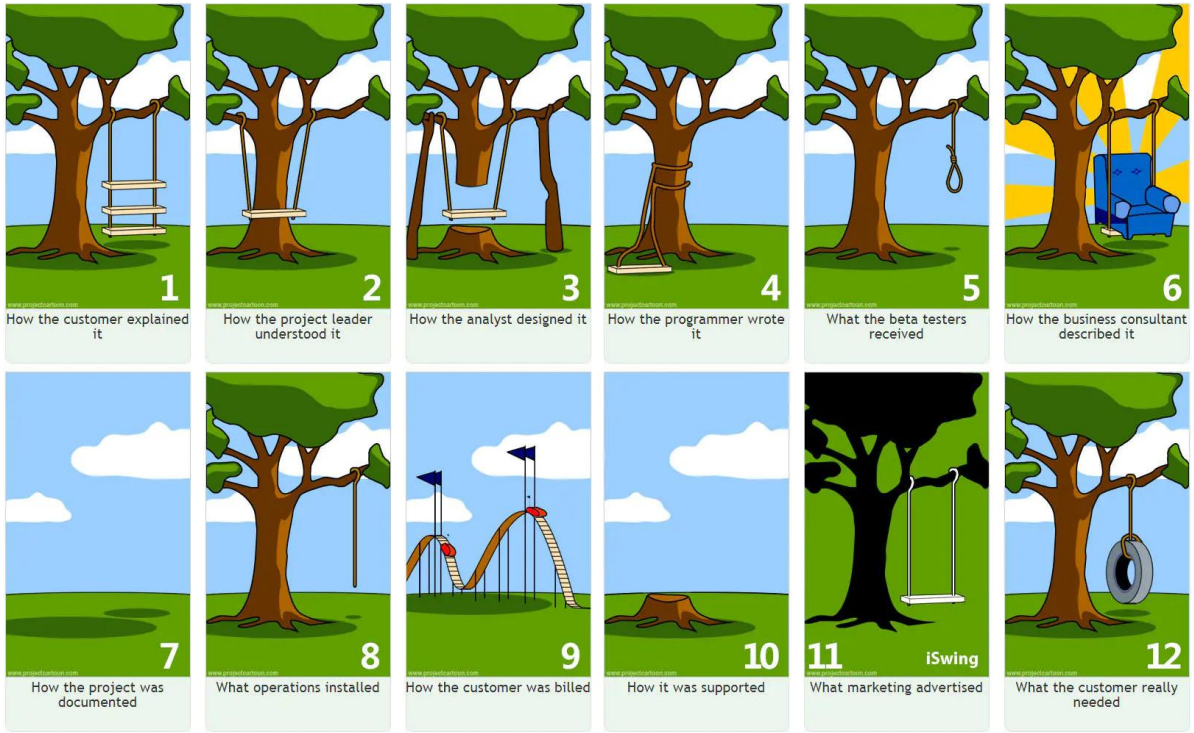



1495 UNIVERSITY OF ABERDEEN

# Software development is difficult!

Building applications is difficult as you are making decisions in face of maximum uncertainty and ambiguity.

# An unsuccessful software development case




This example includes many issues in a software product development process.

Can you figure them out?

# An unsuccessful software development case



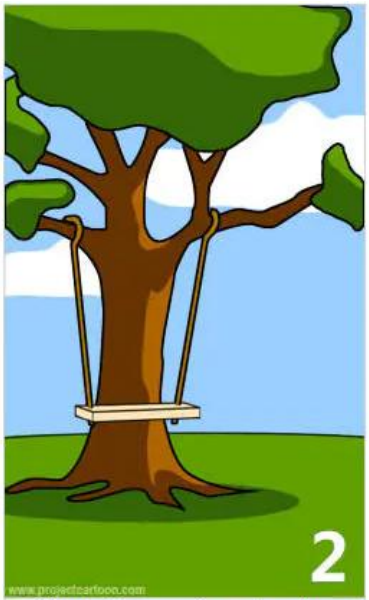
How the customer explained it

**How users explained the software they want.**

“I have three children at home and I need a swing that can be used by three people. It needs to hang from a tree in my garden using a rope.”

Customers tend to provide too much information when describing their needs.

# An unsuccessful software development case




**How the project leader understood it.**

“The swing is too simple. The swing is just a board, which is tied with ropes on both sides and hung on two branches on the tree.”

The project leader figures what the swing looks like but does not know how the product should work.

# An unsuccessful software development case




**How the analyst designed it.**

Engineers design the product according to the requirements of the product manager. How can a swing hanging on two branches still move, unless the tree is truncated and re-branched?

The project architect knows how the product should work and finds the problems described by the project leader, but the problem is not communicated to the project leader or users.

# An unsuccessful software development case

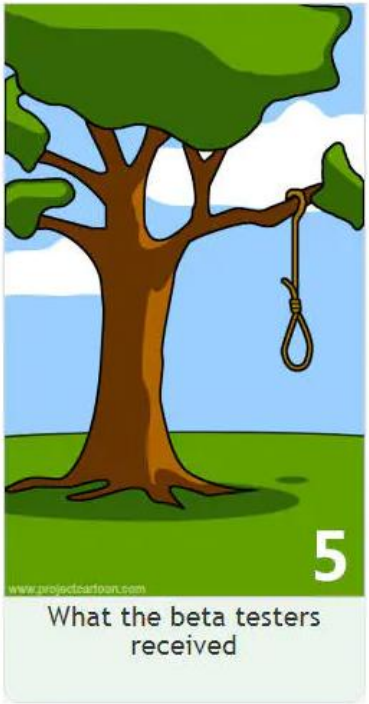



**How the programmer wrote the code.**

Start writing the program. Two ropes, a board, and a big tree are connected to the middle of the trees; it is too simple, and the process is completed.

Programmers complete the product but lack knowledge of the environment in which the product will be deployed.

# An unsuccessful software development case




**What the beta testers received.**

Testers receive products from the development department for testing: a rope with a loop tied at the end.

The setup of unit tests may be too demanding for testers.

# An unsuccessful software development case

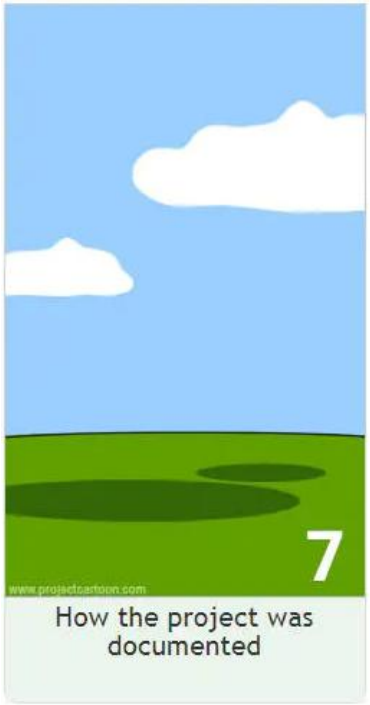



**How the business consultant described it.**

Finally, the product is completed, and the salesman starts to sell to customers. Through ergonomics and engineering mechanics research, and in line with customer service, our swing products make you as comfortable as sitting on a sofa.

Exaggerated propaganda!

# An unsuccessful software development case

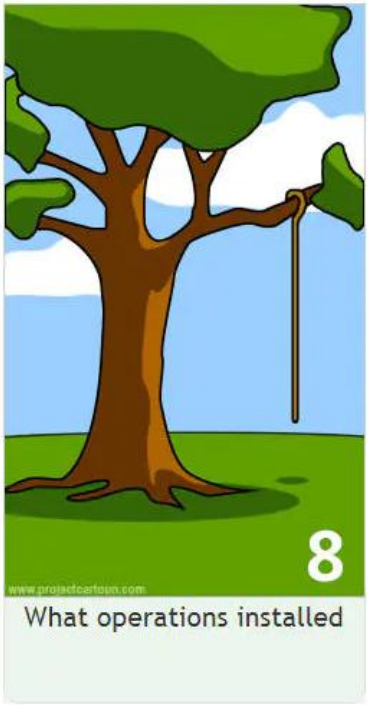



**How the project was documented.**

When you need to use the documentation, you cannot always find it. It is normal for such a project to have no documentation, if the requirements specification and contract are sufficient.

Documentation of the development process is important for the growth of IT companies, and documentation should not be omitted even for small projects.

# An unsuccessful software development case




**What operations installed.**

When implementing the product delivered, simply tie the rope to the tree.

Software description is not detailed!

# An unsuccessful software development case

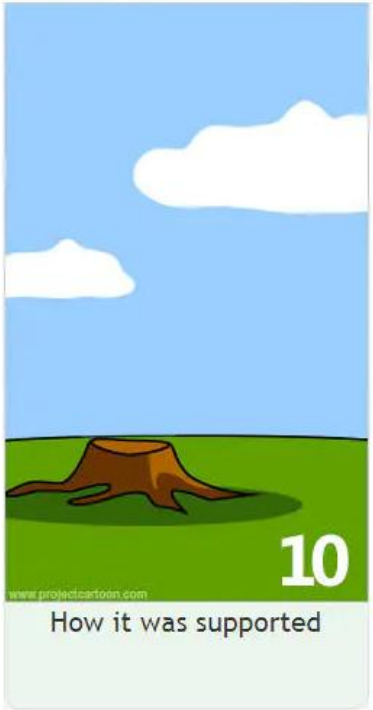



**How the customer was billed.**

A software developed based on the user requirements is usually very expensive, because, generally, the software cannot be sold to other users.

Custom software development is always expensive.

# An unsuccessful software development case




**How it was supported.**

When the seller finds issues in the project, they tend to simply disband the team and cut the project. Many companies maintain very little products and simply cut projects when problems occur.

In large IT companies (e.g. Amazon), each programming team is usually responsible for one project. If the project is cut, most people in the team might be fired.

# An unsuccessful software development case




**What marketing advertised.**

# An unsuccessful software development case




**What the user really needed.**

This is the feature that users really need. Usually, users only need very simple functions, but there is a communication problem. Users always want to add too much functionality to custom software. This is not because they NEED these functions, but because they are not familiar with their own business, and are afraid of missing out on functions which might be necessary.

Therefore, in the requirements analysis phase, software engineers need to identify as much as possible about the core functions of the users' business.

# Challenges of software product development

**Product challenges:**

- How to collect/analyse requirements
- How to architect a system
- How to test the product and ensure the product quality

# Challenges of software product development

**Process challenges:**

- How to ensure you do not overrun your budget
- How to ensure projects are completed on time

# Challenges of software product development

**Human resources challenges:**

- How to deal with personnel risks
- Employee turnover, training new employees, etc.

# Software projects fail

- Unrealistic or unarticulated project goals
- Inaccurate estimates of needed resources
- Badly defined system requirements
- Poor reporting of the project's status
- Unmanaged risks
- Poor communication among customers, developers, and users
- Use of immature technology
- Inability to handle the project's complexity
- Sloppy development practices
- Poor project management
- Stakeholder politics
- Commercial pressures

# How might we fix this?

- How do we build software well so that it does what it should?
- How do we know that we built the *right* software, so that we know it will serve our purpose?

# Course overview, learning objectives, and assessment







UNIVERSITY OF ABERDEEN

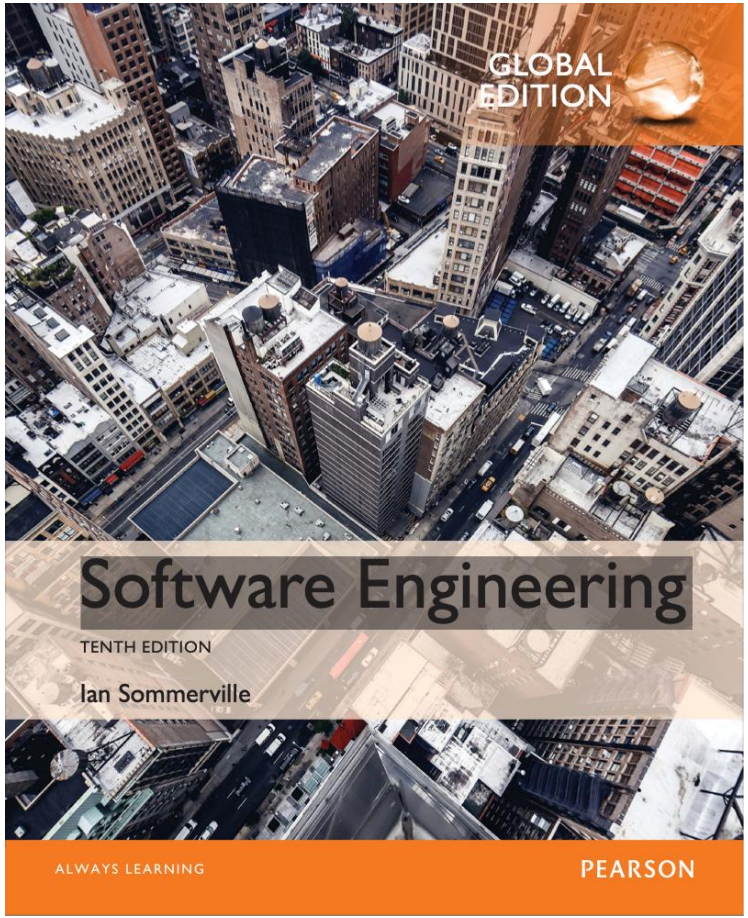
# Course topics

**During the course we will look at a range of topics covering:**

- Software engineering and development in general
- Perspectives on software engineering and development
- Team culture and collaboration
- Version control and development practices
- Testing and deployment
- Software and architectural design
- Security and reliability aspects of design

# Textbook & literature

**Software Engineering, 10th edition, by Ian Sommerville**



# Course learning outcomes

- Apply knowledge of software development paradigms and their role in the software development process.
- Apply knowledge of software design practices and pattern-based design techniques.
- Apply knowledge of quality assurance principles and techniques, with particular emphasis on software testing methodologies and techniques.
- Apply knowledge of project management concepts, methodologies, and tasks.
- Apply knowledge of software requirements elicitation and analysis principles and techniques.

# Assessment

- Final Examination – 50% weightage of the course mark
- Group Project – 50% weightage of the course mark

See the “Assessment and Feedback” tab on the course page
https://www.abdn.ac.uk/registry/courses/undergraduate/2026-2027/scnu_joint_institute_computing_science/jc2001

# Assessment

- Final Examination (50% of the Course Marks) - The examination date will be announced later.
- Group Project (50% of the Course Marks) - **Join a Group** on MyAberdeen on or before **Monday, 14 September 2026 at 23:59 CST (China Standard Time).** **Supervisor(s) allocations** will be announced by **Friday, 18 September 2026.**
  1. **Project proposal –** 0% mark weighting but submission is mandatory (otherwise penalty)**:**
     Submit the project proposal on or before **Friday, 25 September 2026 at 23:59 CST.**
  2. **Technical Report** - 50% of the Group Project Mark
     Submit on or before **Monday, 14 December 2026 at 23:59 CST**
  3. **Proof-of-Concept Software** - 30% of the Group Project Mark
     Submit on or before **Monday, 14 December 2026 at 23:59 CST**
  4. **Presentation** - 20% of the Group Project Mark
     Submit on or before **Monday, 14 December 2026 at 23:59 CST**
  5. **Project Updates** – 0% mark weighting – from four, at least three mandatory (otherwise penalty);
     See the course assessment section for update submission dates on the MyAberdeen course page.

***Note:*** *See the project description and submission requirement document for more details, including the penalties, mark weightings, etc.*

# Group Project – Pick your group

- Diversity trumps homogeneity, so create a diverse group.
- Stay focused on the goals you want from the experience when working with friends.
- Who could you learn the most from?
- You need to be in a group by **Monday, 14 September 2026 at 23:59 CST (China Standard Time)**, for assessment work. Setting up a group as early as possible is recommended so you can start discussing project ideas.
- Self-enrolment within groups (teams) is enabled on the course page on MyAberdeen.

# Assessments Extension Policy

- You should apply for an extension by emailing your request (using your UoA email address) to uoa-ji-enquiries@abdn.ac.uk. Students should also include any supporting evidence where possible e.g. a medical letter.
- Please read the policy on Extensions and Late Submission of Work for full details.

# Plagiarism

“Plagiarism” is defined by the University as the use, without adequate acknowledgment, of the intellectual work of another person in work submitted for assessment. A student cannot be found to have committed plagiarism where it can be shown that the student has taken all reasonable care to avoid representing the work of others as his or her own.

https://www.abdn.ac.uk/staffnet/documents/academic-quality-handbook/Code%20of%20Practice%20in%20Student%20Discipline%20(Academic).pdf

# Plagiarism

- Work should reflect your own intellectual effort.
- Others' work should not be used unless they are an authorised source.
- Assignment setters will determine what are, or are not, authorised sources.

**If in doubt, ask.**

All submissions in JC2001 are submitted to Turnitin for plagiarism checking

# Plagiarism in reports

- If taking text verbatim, put it in quotes and cite the source.
- Alternatively, paraphrase and cite the source.
- Or summarise and cite the source.
- As a rule of thumb, don’t do this for more than 3-4 sentences.
- See https://integrity.mit.edu/handbook/writing-original-work for an excellent guide.

# Collusion & Contract Cheating

**3.3 Collusion**

"Collusion" is a form of plagiarism and is therefore treated in the same way as plagiarism. It is defined as collaboration between students in an assignment that has not been authorised by the course coordinator. It does not refer to authorised group work that is assessed by a single group report.

**3.4 Contract cheating**

"Contract cheating" is an umbrella term to denote the submission of work by a student that has been produced by someone other than that student with the intention to deceive. This can be achieved through having another person, or commercial service, produce work that is subsequently submitted for an assessment, whether that person/commercial service is paid or not.

https://www.abdn.ac.uk/staffnet/documents/academic-quality-handbook/Code%20of%20Practice%20in%20Student%20Discipline%20(Academic).pdf

# Submit your own work

- An excellent guide: http://finditfastlibraryworkshops.pbworks.com/w/page/103649739/Ethical%20Use%20of%20Information
- Never copy other students' work
- Don't simply reuse code or text you find online
- Paraphrase, and then cite the sources of ideas that you use within your work
- Ensure that any licensing conditions are adhered to

# Summary

- We know what a software product is and how it differs from a software prototype.
- We understand why it is important to study software engineering.
- We know what types of challenges are encountered during software development.
- **In the next lecture…** we will learn about software development paradigms.

# Communication

Please reach out when you are struggling during the course:


| | |
|---|---|
| Dr Zhenhua Yu | Zhenhua.yu@abdn.ac.uk |
| Dr Riazul Islam | riazul.islam@abdn.ac.uk |
| Dr Shahzad Mumtaz | shahzad.mumtaz@abdn.ac.uk |